A Practical Guide to CycleGAN

TOPIC -- CYCLEGAN

-- 01 --

If both the generator's and the discriminator's strategy sets are spanned by a finite number of strategies, then by the minimax theorem, min μ G max μ D L (μ G , μ D) = max μ D min μ G L (μ G , μ D) {\displaystyle \min _{\mu _{G}}\max _{\mu _{D}}L(\mu _{G},\mu _{D})=\max _{\mu _{D}}\min _{\mu _{G}}L(\mu _{G},\mu _{D})} that is, the move order does not matter. However, since the strategy sets are both not finitely spanned, the minimax theorem does not apply, and the idea of an "equilibrium" becomes delicate. To wit, there are the following different concepts of equilibrium:

-- 02 --

Concerns have been raised about the potential use of GAN-based human image synthesis for sinister purposes, e.g., to produce fake, possibly incriminating, photographs and videos. GANs can be used to generate unique, realistic profile photos of people who do not exist, in order to automate creation of fake social media profiles. In 2019 the state of California considered and passed on October 3, 2019, the bill AB-602, which bans the use of human image synthesis technologies to make fake pornography without the consent of the people depicted, and bill AB-730, which prohibits distribution of manipulated videos of a political candidate within 60 days of an election. Both bills were authored by Assembly member Marc Berman and signed by Governor Gavin Newsom. The laws went into effect in 2020. DARPA's Media Forensics program studies ways to counteract fake media, including fake media produced using GANs.

-- 03 --

Conditional GAN Conditional GANs are similar to standard GANs except they allow the model to conditionally generate samples based on additional information. For example, if we want to generate a cat face given a dog picture, we could use a conditional GAN. The generator in a GAN game generates μ G {\displaystyle \mu _{G}} , a probability distribution on the probability space Ω {\displaystyle \Omega } . This leads to the idea of a conditional GAN, where instead of generating one probability distribution on Ω {\displaystyle \Omega } , the generator generates a different probability distribution μ G (c) {\displaystyle \mu _{G}(c)} on Ω {\displaystyle \Omega } , for each given class label c {\displaystyle c} . For example, for generating images that look like ImageNet, the generator should be able to generate a picture of cat when given the class label "cat". In the original paper, the authors noted that GAN can be trivially extended to conditional GAN by providing the labels to both the generator and the discriminator. Concretely, the conditional GAN game is just the GAN game with class labels provided: L (μ G , D) := E c ~ μ C , x ~ μ ref (c) ■ [ln ■ D (x , c)] + E c ~ μ C , x ~ μ G (c) ■ [ln ■ (1 − D (x , c))] {\displaystyle L(\mu _{G},D):=\operatorname {E} _{c\sim \mu _{C},x\sim \mu _{\text{ref}}(c)}[\ln D(x,c)]+\operatorname {E} _{c\sim \mu _{C},x\sim \mu _{G}(c)}[\ln(1-D(x,c))]} where μ C {\displaystyle \mu _{C}} is a probability distribution over classes, μ ref (c) {\displaystyle \mu _{\text{ref}}(c)} is the probability distribution of real images of class c {\displaystyle c} , and μ G (c) {\displaystyle \mu _{G}(c)} the probability distribution of images generated by the generator when given class label c {\displaystyle c} . In 2017, a conditional GAN learned to generate 1000 image classes of ImageNet.

-- 04 --

StyleGAN-2 StyleGAN-2 improves upon StyleGAN-1, by using the style latent vector to transform the convolution layer's weights instead, thus solving the "blob" problem. This was updated by the StyleGAN-2-ADA ("ADA" stands for "adaptive"), which uses invertible data augmentation as described above. It also tunes the amount of data augmentation applied by starting at zero, and gradually increasing it until an "overfitting heuristic" reaches a target level, thus the name "adaptive".

-- 05 --

Compared to fully visible belief networks such as WaveNet and PixelRNN and autoregressive models in general, GANs can generate one complete sample in one pass, rather than multiple passes through the network. Compared to Boltzmann machines and linear ICA, there is no restriction on the type of function used by the network. Since neural networks are universal approximators, GANs are asymptotically consistent. Variational autoencoders might be universal approximators, but it is not proven as of 2017.

-- 06 --

Choice of the strategy set In the most generic version of the GAN game described above, the strategy set for the discriminator contains all Markov kernels μ D : Ω → P [0 , 1] {\displaystyle \mu _{D}:\Omega \to {\mathcal {P}}[0,1]} , and the strategy set for the generator contains arbitrary probability distributions μ G {\displaystyle \mu _{G}} on Ω {\displaystyle \Omega } . However, as shown below, the optimal discriminator strategy against any μ G {\displaystyle \mu _{G}} is deterministic, so there is no loss of generality in restricting the discriminator's strategies to deterministic functions D : Ω → [0 , 1] {\displaystyle D:\Omega \to [0,1]} . In most applications, D {\displaystyle D} is a deep neural network function. As for the generator, while μ G {\displaystyle \mu _{G}} could theoretically be any computable probability distribution, in practice, it is usually implemented as a pushforward: μ G = μ Z ■ G − 1 {\displaystyle \mu _{G}=\mu _{Z}\circ G^{-1}} . That is, start with a random variable z ∼ μ Z {\displaystyle z\sim \mu _{Z}} , where μ Z {\displaystyle \mu _{Z}} is a probability distribution that is easy to compute (such as the uniform distribution, or the Gaussian distribution), then define a function G : Ω Z → Ω {\displaystyle G:\Omega _{Z}\to \Omega } . Then the distribution μ G {\displaystyle \mu _{G}} is the distribution of G (z) {\displaystyle G(z)} . Consequently, the generator's strategy is usually defined as just G {\displaystyle G} , leaving z ∼ μ Z {\displaystyle z\sim \mu _{Z}} implicit. In this formalism, the GAN game objective is L (G , D) := E x ∼ μ ref ■ [ln ■ D (x)] + E z ∼ μ Z ■ [ln ■ (1 − D (G (z)))] . {\displaystyle L(G,D):=\operatorname {E} _{x\sim \mu _{\text{ref}}}[\ln D(x)]+\operatorname {E} _{z\sim \mu _{Z}}[\ln(1-D(G(z)))].}